RIT HMS — System Overview

RIT HMS — System Overview

What it is: a multi-tenant, cloud-native rewrite of the legacy SMART_HMS hospitality management system (POS + back office for Sri Lankan restaurants/hotels), rebuilt as **ASP.NET Core 8 + PostgreSQL + Next.js 15**.

Status: feature-parity-plus with the legacy for day-to-day operations. Live test deployment at **https://hms.retailit.lk**.

*Compiled from the current code surface (49 SQL migrations, 22 API feature modules, ~21 web screens). Items are graded ✅ **built**, 🟡 **v1 / partial**, ⬜ **not built**, 🚫 **won't-do**.*

---

1. Feature inventory

1.1 Point of Sale (POS)

Feature	Status
Order/tab lifecycle (open → confirm → settle/void), resume open tabs, deep-link	✅
Touch terminal: category/product grid, search (OS keyboard), keypad, open/custom items	✅
Modifiers / add-ons, serving-size variants, per-product kitchen routing	✅
Bill discounts (amount / %) + **role-capped** with **manager-PIN override**	✅
Multi-tender settlement: cash, card, Uber Eats / PickMe prepaid, **credit (A/R)**, **loyalty points**	✅
Tables / floor: per-outlet tables, occupancy, table picker, reservations	✅
Split / merge bills + table transfer + **item-level split**	✅
Full-screen mode, sign-out, branch switch (admin)	✅
Receipt **print** (80mm) + **reprint** of a settled bill	✅
Covers (adults/children)	🟡
Steward/waiter tips, multi-currency tender, tour-agent commission	⬜

1.2 Kitchen Display (KDS / KOT)

Feature	Status
Live ticket board by station (Hot Kitchen / Bar / Dessert) + counts	✅
New → Preparing → Ready → Served lifecycle, overdue timers, recall-last-10, PAID badge	✅
Printer-station routing per product	✅
All-day item counts, course firing, expo screen, prep-time analytics	⬜

1.3 Menu & catalogue

Products CRUD + CSV import, multi-level categories, units of measure (+ conversion), modifiers, serving-size variants, price levels, per-location pricing, per-product tax class. ✅

1.4 Tax / VAT (Sri Lanka)

Configurable compound charge engine (Service Charge → SSCL → VAT, per-order-type), compliant tax invoices (sequential numbering, BR/VAT/SVAT, bill branding), VAT-return report. ✅

1.5 Supply chain

Suppliers, Purchase Orders (approval workflow, currency/terms), GRN (purchase-unit→stock-unit conversion, line discounts/landed cost, free items, supplier invoice no., void/reverse), weighted-average costing, supplier returns (PRN + input-VAT reversal), inter-location transfers, wastage, stock adjustments, **physical stock count** (sheet → variance → on-hand). ✅

1.6 Production

Recipes/BOM with UoM conversion, custom/ad-hoc production, draft→post→void, multi-product notes, multiple recipes per output unit, inter-location production, weighted-avg cost roll-up. ✅

1.7 Delivery aggregators

Uber Eats + PickMe integration built as a **swappable mock** (incoming queue, accept/lifecycle, 86 items, per-location config). Real partner sandboxes pending credentials. 🟡

1.8 CRM / customers

Customer master + categories, per-customer (or category) default discount, **credit customers (A/R)** with limit + balance + AR receipts + ledger, visit history, per-product customer pricing, advance/suspended payments. ✅

1.9 Loyalty

Earn points on settle (configurable rate), redeem as a “Points” tender, balance + transaction ledger, **tiers** (earn multiplier + tier discount), **points expiry**, **loyalty cards** (by-card lookup). Expiry-reminder *delivery* needs a notifications channel (follow-on). ✅

1.10 Promotions

Schedulable (date / day / happy-hour window), order-type scoped, **product-discount**, **bill-value (spend & save)**, auto-applied + snapshotted; BOGO/bundle/bank-BIN. ✅

1.11 Reporting

VAT return, sales summary (KPIs / per-day / top items), HQ outlet rollup, **sales register**, **item sales**, **stock balance**, **shift settlement**, **promotion usage**, **food costing** (dish cost vs sell price + GP%), **bin card** (per-product movement ledger), **budget vs sales** (per-outlet monthly). ✅ *(a unified* `inventory_movements` *table to simplify the bin card is the only follow-on)*

1.12 Shifts & cash control

Open/close cashier shifts, opening float, cash-up **Z-report** + variance, billing gated on an open shift, end-shift reconciliation of open bills, **month-end period lock**. ✅

1.13 People, access & audit

Users + 5 RBAC roles (Owner/Manager/Cashier/Kitchen/Accountant), **granular role permissions** (max-discount %, void/comp), **function-level (per-screen) access**, **manager-PIN override**, append-only **activity/audit log** (who did what, when). ✅

1.14 Authentication

Magic-link (email) **and staff PIN login** (username + PIN, PBKDF2-hashed, lockout) for emailless POS staff; rotating JWT + refresh tokens; stale/closed-tenant tokens → 401 (re-auth). ✅

1.15 Platform

Multi-tenant SaaS (control plane + DB-per-tenant + RLS), **tenant auto-provisioning** at signup, configurable document prefixes/branding, multi-outlet (HQ + N locations, branch switching). ✅

1.16 Printing

Print Agent design + print-jobs queue for venue receipt/KOT printers. 🟡

Not built (genuine gaps)

- ⬜ **#67 Gift cards / vouchers** — voucher master, issue/redeem, spend-get-voucher.
- ⬜ **#73 GL / accounting export** — journal posting by cost-centre, AP/supplier payments, expenses, petty cash (beyond the VAT return).
- ⬜ **#75 Catering / banquet** — rooms/rates, events, meal types, own-fleet delivery.
- ⬜ **#74 Handheld waiter app** — a dedicated installable tablet app (a `tab-ordering` web screen exists; native/PWA packaging is pending) and **#57** GRN mobile PWA.
- 🚫 **Serial / warranty tracking** — intentionally out of scope (not F&B).

---

2. System architecture

Key decisions - **DB-per-tenant + RLS.** Each tenant has its own database (`hms_tenant_<slug>`); a control DB (`hms_control`) holds tenant/billing rows + refresh tokens. `TenantDbContextFactory` reads the `tenant_id` JWT claim, looks the tenant up in control, and builds a per-request connection from a template (`{tenant_db}` placeholder). Defence-in-depth: Postgres **RLS** (`app.tenant_id`) **and** an EF global query filter (`TenantId == current && !IsDeleted`). - **Minimal-API feature modules.** Each domain area is a `Map…Endpoints()` extension; services (`OrderService` , `ShiftService` , `PermissionService` , …) hold the logic. - **Auth.** Magic-link (email) or staff PIN (username + PBKDF2 PIN, attempt-lockout); both mint an HS256 JWT + a rotating refresh token. Authorization policies: `Owners` , `BackOffice` , `SupplyChain` , `Operations` , `KitchenView` , `PlatformAdmin` . - **Money/tax integrity.** Settlement decrements stock + records payment in one transaction; the compound tax engine + VAT invoice numbering are server-side and configurable. - **Tenant provisioning.** Signup runs the migration set + a baseline seed against a fresh tenant DB.

Tech stack | Layer | Tech | |—|—| | Web | Next.js 15 (App Router, React, TypeScript, Tailwind — Stitch design system) | | API | ASP.NET Core 8 Minimal API, EF Core 8, Npgsql | | DB | PostgreSQL 16 (snake_case), DB-per-tenant + RLS, raw SQL migrations (0001–0049) | | Auth | JWT (HS256) + rotating refresh, magic-link, staff PIN (PBKDF2) | | Tests | xUnit integration (real Postgres) + Vitest + Playwright E2E + CI |

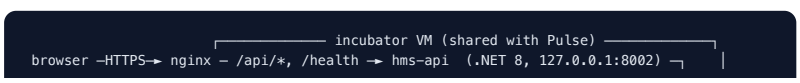
---

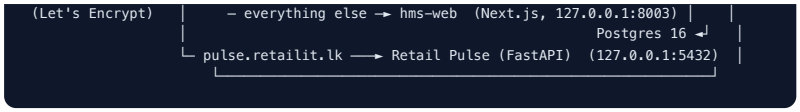
3. Deployment architecture

Two shapes are documented (full runbook: `docs/deploy.md`).

A) Incubator — bare-metal (LIVE today)

URL: `https://hms.retailit.lk` · shared RIT incubator VM (Ubuntu, alongside Retail Pulse). System Postgres + nginx + systemd + Let's Encrypt (certbot).





- `hms-api` and `hms-web` run as **systemd services** bound to localhost; **nginx** terminates TLS and reverse-proxies by path; **Postgres 16** is local (control DB + one DB per tenant).
- ⚠️ Unrelated to the legacy RIT servers `MF-SW-LP24` / `161.97.172.35` — those are never touched.

B) Portable — Docker Compose (own server / staging)

Self-contained stack in `docker-compose.yml`: **postgres** → **bootstrap** (one-shot: create + migrate `hms_control`) → **api** (ASP.NET Core 8) → **web** (Next.js) → **caddy** (reverse proxy + auto-TLS). Secrets via `./.env` (`infra/gen-secrets.sh`). Brings HMS up on a dedicated box with one command.

C) Local development

`make dev` runs the API (`:5000`) + web (`:3000`) against a local Postgres. `make reset` drops/recreates + applies all migrations + seeds the demo tenant; `make seed-demo` adds demo customers/promotions/PO+GRN/orders/PINs/loyalty.

Secrets posture: JWT signing key + master encryption key come from the environment/secret store (prod fails fast if the secret is the dev default); aggregator merchant credentials are AES-GCM encrypted in the DB (not env).

---

4. Quality

- **Integration tests** run against a real ephemeral Postgres (control + tenant DBs, real migrations, RLS) through the actual HTTP pipeline — ~198+ green.
- **Unit** (Vitest) + **E2E** (Playwright) + **CI**.
- Append-only **audit log** records sensitive actions (settle, void, discount, shift open/close, permission/PIN changes, stock-count post) with the acting user + role.

---

5. Outstanding (not code)

- **Rotate the leaked legacy SA passwords** on the live RIT servers (highest priority, ops action).
- Apply for **Uber Eats + PickMe partner sandboxes** (integration is mock-ready).
- Internal sign-off on repo destination · pilot/reconciliation with a real outlet.

See `docs/backlog.md` for the detailed legacy→new parity register and the remaining items (#67 gift cards, #73 GL export, #75 catering, and the v1 follow-ons).